

Project Scope: Movie Intelligence Platform

1. Purpose

A portfolio project and personal learning vehicle. The goal is one working system, built end to end, where each technology is used because the system needs it.

2. Project definition

An analytical platform for understanding movies and audience behaviour, with recommendation capabilities. It is **not** "a machine-learning project about movies."

Dataset: [The Movies Dataset](#) (Kaggle): movies_metadata, credits, keywords, links, ratings.

3. Core questions the platform answers

1. **Movie economics:** How do budget, revenue, profit and ROI relate? What drives ROI?
2. **Audience behaviour:** How are ratings distributed? How do popularity and rating count relate to ratings?
3. **Industry analysis:** How have genres, directors, cast and languages changed over time?
4. **Recommendation:** Given a movie or user, which movies should be recommended?

4. In scope (must-haves)

Area	Deliverable
Data	Raw data understood, with a data dictionary and data-quality report
ETL	Reproducible Python pipeline; raw data never modified
Database	Normalized PostgreSQL schema with keys, constraints and indexes
SQL analytics	Non-trivial analytical queries (CTEs, window functions)
Statistical analysis	Defensible conclusions with stated limitations
Recommendation system	Content-based first, then collaborative filtering

Area	Deliverable
Analytics layer	Reusable Python modules, not notebook-only code
API	FastAPI serving movie data, analytics and recommendations
Dashboard	Streamlit app usable by someone who has never seen the code
Deployment	Dockerized and deployed on AWS (EC2, RDS, S3, IAM, CloudWatch)
Testing	pytest for ETL, analytics and API; data-integrity checks
Documentation	README with architecture, ER diagram, screenshots, decisions, limitations

5. Stretch goals (only if time allows)

- Revenue prediction model (linear regression, Random Forest, gradient boosting)
- Hybrid recommender (content-based + collaborative)
- "Break it on purpose" troubleshooting exercises, documented

6. Out of scope

- Real-time or streaming data
- User accounts, authentication or a production-grade user system
- Adding technologies beyond the listed stack
- Learning AWS services beyond EC2, RDS, S3, IAM and CloudWatch
- Scraping or merging external datasets

7. Success criteria

- docker compose up runs the full application on a fresh machine
- The app is publicly reachable on AWS, with no hardcoded secrets
- A stranger can use the dashboard and understand the README without help
- I can explain every schema, modeling and design decision without looking at the code

8. Timeline

- **Target finish:** April 2027
- **Hard deadline:** August 2027 (about 4 months of buffer)

Suggested pacing (starting October 2026; adjust to your university workload):

Period	Focus
Oct to Nov 2026	Scope, data understanding, ETL (M0 to M2)
Nov to Dec 2026	PostgreSQL design and loading, SQL analytics (M3 to M4)
Jan 2027	Statistical analysis (M5)
Feb 2027	Recommendation system (M6)
Mar 2027	Analytics layer, FastAPI, Streamlit (M7 to M9)
Apr 2027	Docker, AWS, testing, documentation (M10 to M14)

9. Priorities when time is short

SQL → PostgreSQL → Python/data → statistics → recommender → API → Docker → AWS → testing. Drop stretch goals first, then cut depth in later stages, rather than spreading effort evenly.

10. Risks and limitations to note

- Many budget and revenue values are 0 (missing, not real zeros), which limits economic analysis.
- Some records are malformed, and genres, cast, crew and keywords are stored as stringified JSON.
- The full 26M-row ratings file is heavy; ratings_small.csv may be used for development.
- Correlation is not causation; confounders will be documented.